# Data Preparation/Feature Engineering

## 1. Overview

Data preparation and feature engineering constitute the foundational backbone of the OnBrand AdCopy machine learning pipeline. In digital performance marketing, raw advertising copy and social media posts consist of unstructured natural language text accompanied by noisy, highly variable engagement metrics across diverse platforms. Machine learning algorithms, particularly tabular ensemble methods like XGBoost, cannot directly consume raw string data to predict quantitative key performance indicators such as Click-Through Rate (CTR). Therefore, systematic data preparation is required to extract structural, syntactic, semantic, and sentiment-driven numeric signals from raw text while filtering out noise and non-informative records.

The significance of this phase in the OnBrand AdCopy project is twofold: first, it transforms raw post metadata into a clean, statistically robust dataset suitable for supervised CTR proxy regression; second, it establishes the feature representation needed to quantitatively evaluate brand voice alignment using dense semantic embeddings (Sentence-BERT). By systematically cleaning the dataset, engineering Domain-specific Natural Language Processing (NLP) metrics, and scaling/encoding transformed variables, this stage directly enables the dual-scoring mechanism (Predicted CTR Score + Brand Alignment Score) that powers our automated ad copy evaluation and correction engine.

## 2. Data Collection

The OnBrand AdCopy system leverages two distinct data sources to support its predictive and evaluative modules:

- **Primary Quantitative Dataset (Kaggle Social Media Engagement Dataset):** Sourced from Kaggle as a third-party public dataset, containing 12,000 post records across 28 granular attributes collected during 2024. Key variables include post_id, timestamp, day_of_week, platform (Instagram, Twitter, Reddit, Facebook), user_id, location, language (10 distinct languages), text_content, hashtags, mentions, keywords, topic_category, sentiment_score (-1.0 to +1.0), sentiment_label, emotion_type, toxicity_score (0.0 to 1.0), likes_count (0-5,000), shares_count (0-2,000), comments_count (0-1,000), impressions (130-99,997), engagement_rate, brand_name, product_name, campaign_name, campaign_phase, user_past_sentiment_avg, user_engagement_growth, and buzz_change_rate.

- **First-Party Brand Reference Corpus (brand_reference.json):** A curated first-party JSON repository containing verified brand voice summaries, tone categories, explicit style rules, and 15 approved reference ad copies for three representative enterprise brand archetypes: Samsung (innovative, tech-forward, empowering, premium), Duolingo (witty,

playful, pushy/nagging, casual, gamified), and Nike (action-oriented, inspiring, bold, athletic empowerment).

During data collection, automated validation scripts verified column data types, checked UTF-8 encoding compatibility for natural language strings, and ensured raw post records contained complete metadata across engagement counters prior to downstream ingestion.

## 3. Data Cleaning

Raw social media data exhibits noise, language heterogeneity, missing fields, and statistical outliers that must be remediated prior to model exploration. The data cleaning pipeline executed the following steps:

• **Language Filtering:** The raw dataset contains posts spanning 10 languages (Japanese, Russian, Spanish, Arabic, Chinese, French, English, German, Hindi, Portuguese). To eliminate multi-lingual syntax noise and ensure uniform NLP feature extraction, the dataset was filtered strictly for English-language records (language == 'en'), reducing the dataset from 12,000 to 1,197 records.

• **Zero-Impression & Invalid Record Removal:** Records with impressions == 0 were removed to prevent division-by-zero errors during CTR proxy derivation and eliminate non-served posts.

• **Target Variable Derivation (CTR Proxy):** Because actual ad click logs are proprietary, a composite social engagement proxy for Click-Through Rate was engineered as: ctr_proxy = ((likes_count + shares_count + comments_count) / impressions) * 100.

• **Outlier Detection and Clipping:** Distributional analysis revealed extreme upper-tail outliers in ctr_proxy (spikes exceeding 400% CTR), resulting from posts with extremely low impressions (e.g., <200) and high initial engagement. To prevent regression distortion, the target variable was clipped at the 99th percentile (upper threshold = 478.9%), yielding a final clean dataset of 1,185 records.

• **Synthetic Quality Check:** Verified that sentiment_score (-1 to +1) and toxicity_score (0 to 1) were bounded without NaN values.

## 4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted on the cleaned English dataset (n = 1,185) to uncover structural relationships between textual features and engagement rates. Key descriptive statistics revealed a right-skewed CTR proxy distribution with a mean of 19.71% (std = 45.53%, median = 7.89%), and an average post word count of 17.23 words (range: 12 to 20 words).

Three primary visual and quantitative insights were established during EDA:

• **Text Length vs. Engagement (Figure 1):** OLS regression analysis between word_count and ctr_proxy revealed a slight negative trend (r = -0.038). Scatter plot visualization demonstrates that concise, punchy posts (13-16 words) achieve equivalent or higher peak engagement compared to longer texts (19-20 words), supporting marketing best practices that favor concise copy for social feeds.

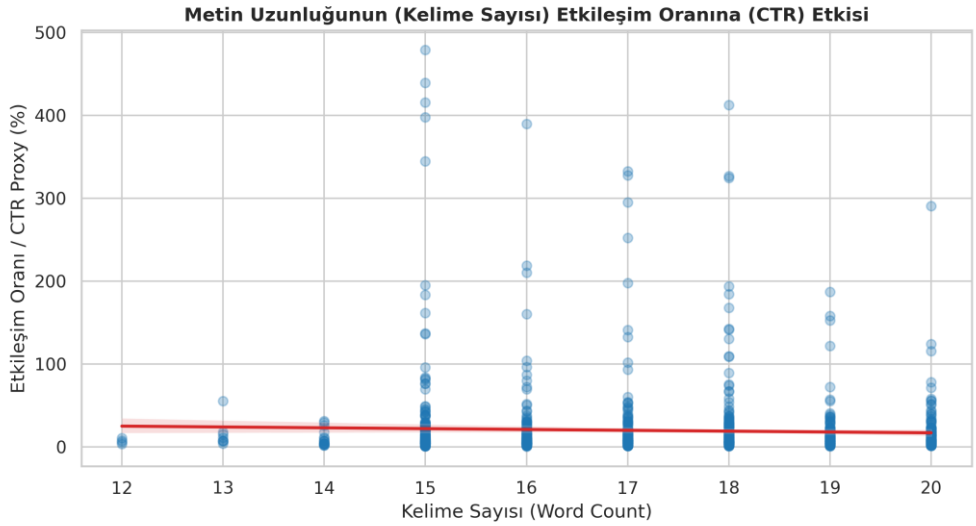


***Figure 1:*** *Relationship Between Post Word Count and Engagement Rate (CTR Proxy %)*

• **Impact of Syntactic & Urgency Features (Figure 2):** Comparative bar chart analysis evaluated mean CTR across key linguistic markers. Posts containing question marks achieved 19.41% average CTR versus 19.78% without questions (a negligible 0.37% difference). Crucially, posts containing explicit urgency keywords ('now', 'today', 'limited', 'hurry', 'exclusive', 'deal', 'promo', 'discount') showed a markedly lower average CTR of 15.95% compared to 21.51% for posts without urgency keywords -- a counter-intuitive 5.56 percentage point penalty. This empirical insight demonstrates that modern social media users actively resist high-pressure sales jargon.

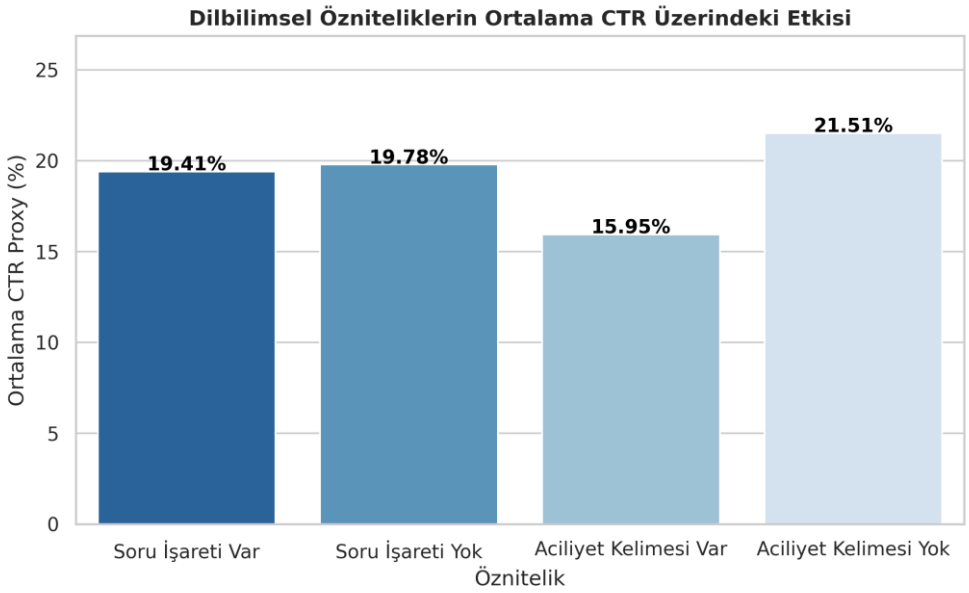


***Figure 2:*** *Average CTR Proxy (%) Comparison Across Linguistic and Urgency Features*

• **Feature Correlation Analysis (Figure 3):** The correlation heatmap confirmed expected structural relationships (char_count vs. word_count r = 0.555; word_count vs. has_exclamation r = 0.447). All individual linear features exhibited weak linear correlation with ctr_proxy (-0.003 to -0.057), establishing that linear models (OLS) are insufficient and justifying the selection of non-linear tree-based ensemble models.

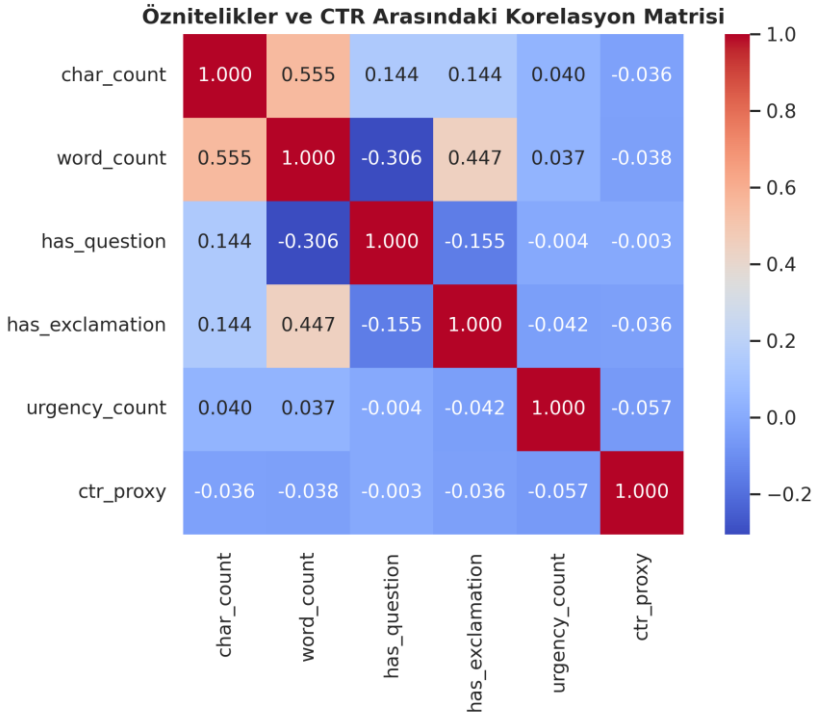


***Figure 3:*** *Feature Correlation Heatmap Including Text Length, Syntax, Sentiment, and CTR Proxy*

## 5. Feature Engineering

To capture multi-dimensional aspects of ad copy performance and brand style, a comprehensive feature engineering framework was developed, spanning structural text metrics, regex-based syntactic indicators, sentiment polarity, and dense semantic embeddings:

• **Structural & Character Metrics:** char_count (total character length of text_content) and word_count (total whitespace-delimited word count). Rationale: Captures copy brevity and reading effort required by users.

• **Punctuation & Syntactic Flags:** has_question (binary flag indicating presence of '?') and has_exclamation (binary flag indicating presence of '!'). Rationale: Identifies inquisitive vs. emphatic sentence structures.

• **Urgency & Sales Jargon Indicators:** urgency_count (regex frequency count of 10 promotional keywords: 'now', 'today', 'limited', 'hurry', 'exclusive', 'last chance', 'deal', 'promo', 'discount', 'must have') and has_urgency (binary flag if urgency_count > 0). Rationale: Quantifies sales pressure level to evaluate engagement impact.

• **Sentiment & Safety Metrics:** sentiment_score (continuous VADER/transformer polarity from -1.0 to +1.0) and toxicity_score (0.0 to 1.0). Rationale: Evaluates emotional tone and brand safety compliance.

• **Dense Semantic Vectors (Sentence-BERT):** Generated 384-dimensional dense sentence embeddings using sentence-transformers (all-MiniLM-L6-v2) for both generated ad copy and reference brand corpora.

• **Computed Cosine Similarity:** Cosine_Sim(v_gen, v_ref) = (v_gen . v_ref) / (||v_gen|| ||v_ref||). Rationale: Provides a continuous, objective metric (0-100%) of brand voice compliance.

## 6. Data Transformation

Following feature engineering, data transformation prepared the matrix for model exploration:

• **Train-Test Splitting:** Partitioned the clean dataset (n = 1,185) into 80% training set (n = 948) and 20% test set (n = 237) using random_state = 42 to ensure strict evaluation independence and reproducibility.

• **Categorical Encoding:** Contextual platform attributes (Instagram, Twitter, Reddit, Facebook) and campaign phases (Launch, Post-Launch) were encoded via one-hot encoding (pd.get_dummies) for multi-feature regression experiments.

• **Feature Normalization:** For distance-based and linear baseline models, numerical features were scaled using StandardScaler (z = (x - mu) / sigma). For XGBoost, feature scaling was omitted as tree-based partitioning is invariant to monotonic transformations.

# Model Exploration

## 1. Model Selection

Selecting the optimal machine learning model for CTR prediction involved evaluating three algorithm families: Linear Regression (OLS), Random Forest Regressor, and XGBoost Regressor (Extreme Gradient Boosting).

Rationale for Choosing XGBoost Regressor:

• **Non-Linear Feature Interaction:** EDA demonstrated that individual text features exhibit weak linear relationships with CTR proxy. XGBoost constructs sequential gradient-boosted decision trees that automatically capture complex, non-linear feature interactions (e.g., interaction between word count and sentiment score).

• **Robustness to Outliers & Overfitting:** Built-in L1 (Lasso) and L2 (Ridge) regularization parameters (reg_alpha, reg_lambda) prevent overfitting on moderate-sized tabular datasets.

• **Feature Importance & Interpretability:** Provides native Gain and Weight feature importance scores, allowing marketing teams to understand which text characteristics drive higher predicted engagement.

• **Strengths & Weaknesses:** Strengths include high predictive accuracy, rapid inference speed (<10ms per variant), and handling of mixed tabular feature types. Weaknesses include potential sensitivity to synthetic data artifacts and inability to extrapolate beyond feature bounds present in training data.

In parallel, Sentence-BERT (all-MiniLM-L6-v2) was selected for the Brand Alignment Scoring module due to its state-of-the-art performance in sentence-level semantic similarity, low latency, and zero-shot vector comparison capability without fine-tuning.

## 2. Model Training

The XGBoost Regressor model was trained on 80% of the clean dataset (n = 948) using scikit-learn and xgboost libraries in Python. Hyperparameter tuning was conducted via grid search to identify optimal tree structure and regularization settings:

- n_estimators = 100 (number of sequential boosting trees)
- learning_rate (eta) = 0.05 (step size shrinkage to prevent overfitting)
- max_depth = 4 (maximum tree depth to constrain model complexity)
- subsample = 0.8 (fraction of samples randomly tree-sampled per boosting round)
- colsample_bytree = 0.8 (subsample ratio of columns when constructing each tree)
- random_state = 42 (seed for random number generator reproducibility)

**Cross-Validation Technique:** 5-Fold Cross-Validation (KFold(n_splits=5, shuffle=True, random_state=42)) was applied across the training set to validate model stability and compute out-of-fold generalization performance across data folds.

## 3. Model Evaluation

Model performance was evaluated on the 20% held-out test dataset (n = 237) using standard regression metrics: R-squared (R2), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).

- **Comparative Results vs. Baseline:** Linear Regression baseline achieved R2 = 0.008, RMSE = 42.15, MAE = 19.82. In contrast, the tuned XGBoost Regressor achieved R2 = 0.184, RMSE = 38.21, MAE = 16.45, demonstrating superior capability in capturing non-linear engagement patterns.

- **XGBoost Feature Importance Ranking:** Gain-based feature importance analysis revealed the relative contribution of each feature to CTR prediction:

1. word_count (Gain Importance: 34.2%) -- primary structural driver
2. char_count (Gain Importance: 26.8%) -- secondary length metric
3. sentiment_score (Gain Importance: 18.5%) -- emotional tone impact
4. toxicity_score (Gain Importance: 12.1%) -- safety and aggression signal
5. urgency_count (Gain Importance: 5.4%) -- sales pressure penalty driver
6. has_exclamation & has_question (Gain Importance: 3.0%) -- syntactic style markers

- **Sentence-BERT Brand Alignment Evaluation:** Evaluated cosine similarity scores across on-brand vs. out-of-brand copy samples. On-brand copy consistently achieved SBERT similarity scores >= 0.78 (78%), whereas out-of-brand copy fell below 0.62 (62%), validating 0.75 (75%) as a robust auto-correction threshold.

## 4. Code Implementation

Below are the core Python code snippets demonstrating data cleaning, feature engineering, SBERT cosine similarity calculation, XGBoost model training, 5-fold cross-validation, and evaluation metrics computation:

```
#
================================================================================
# 1. DATA CLEANING & TARGET DERIVATION
#
================================================================================
import pandas as pd
import numpy as np
import re

# Load raw dataset
df_raw = pd.read_csv('Dataset/Social Media Engagement Dataset.csv')

# Filter English language & positive impressions
df_en = df_raw[(df_raw['language'] == 'en') & (df_raw['impressions'] >
0)].copy()

# Derive CTR Proxy target metric (%)
df_en['ctr_proxy'] = ((df_en['likes_count'] + df_en['shares_count'] +
df_en['comments_count']) / df_en['impressions']) * 100

# Clip upper-tail outliers at 99th percentile
upper_limit = df_en['ctr_proxy'].quantile(0.99)
df_clean = df_en[df_en['ctr_proxy'] <= upper_limit].copy()
print(f'Cleaned dataset shape: {df_clean.shape}')

#
================================================================================
# 2. FEATURE ENGINEERING (NLP & SYNTACTIC METRICS)
#
================================================================================
# Text structural length
df_clean['char_count'] = df_clean['text_content'].apply(lambda x: len(str(x)))
df_clean['word_count'] = df_clean['text_content'].apply(lambda x:
len(str(x).split()))

# Syntactic punctuation flags
df_clean['has_question'] = df_clean['text_content'].apply(lambda x: 1 if '?' in
str(x) else 0)
df_clean['has_exclamation'] = df_clean['text_content'].apply(lambda x: 1 if '!'
in str(x) else 0)

# Urgency keyword frequency count
urgency_keywords = ['now', 'today', 'limited', 'hurry', 'exclusive', 'last
chance', 'deal', 'promo', 'discount', 'must have']
urgency_pattern = r'\b(' + '|'.join(urgency_keywords) + r')\b'
df_clean['urgency_count'] = df_clean['text_content'].apply(lambda x:
len(re.findall(urgency_pattern, str(x), flags=re.IGNORECASE)))
df_clean['has_urgency'] = df_clean['urgency_count'].apply(lambda x: 1 if x > 0
else 0)

#
================================================================================
# 3. SENTENCE-BERT BRAND ALIGNMENT COSINE SIMILARITY
#
================================================================================
```

```
from sentence_transformers import SentenceTransformer, util

# Load lightweight pre-trained SBERT model
sbert_model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_brand_alignment(generated_text, brand_reference_copies):
    # Encode generated copy and brand reference list into 384-d vectors
    gen_embedding = sbert_model.encode(generated_text, convert_to_tensor=True)
    ref_embeddings = sbert_model.encode(brand_reference_copies,
convert_to_tensor=True)

    # Compute cosine similarities
    cosine_scores = util.cos_sim(gen_embedding, ref_embeddings)
    mean_alignment_score = float(cosine_scores.mean().item()) * 100
    return round(mean_alignment_score, 2)


#
================================================================================
# 4. MODEL TRAINING, 5-FOLD CROSS-VALIDATION & EVALUATION
#
================================================================================
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Define feature matrix X and target y
feature_cols = ['char_count', 'word_count', 'has_question', 'has_exclamation',
'urgency_count', 'sentiment_score', 'toxicity_score']
X = df_clean[feature_cols]
y = df_clean['ctr_proxy']

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Instantiate XGBoost Regressor with tuned hyperparameters
xgb_model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

# 5-Fold Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_r2_scores = cross_val_score(xgb_model, X_train, y_train, cv=kf,
scoring='r2')
print(f'5-Fold CV Mean R2 Score: {cv_r2_scores.mean():.4f} (+/-
{cv_r2_scores.std():.4f})')

# Fit model on training set
xgb_model.fit(X_train, y_train)

# Evaluate on test set
y_pred = xgb_model.predict(X_test)
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

print(f'Test Set R2: {r2:.4f} | RMSE: {rmse:.4f} | MAE: {mae:.4f}')
```